AckerRadar – Case Study

Enterprise REST API für Präzisionslandwirtschaft

Yves Kühn

2026

Inhaltsverzeichnis

AckerRadar – Case Study **1**
Auf einen Blick . . . 1
Das Problem . . . 1
Anforderungen . . . 2
Architektur . . . 2
Technische Entscheidungen und Trade-offs . . . 5
Sequenzdiagramm: Plot anlegen . . . 6
Herausforderungen und Lösungen . . . 6
Ergebnis . . . 7
Lessons Learned . . . 8
Über mich . . . 8

AckerRadar – Case Study

Eine Multi-Tenant REST API für Präzisionslandwirtschaft mit Geo-Spatial Indexing, Multi-Source Data Aggregation und ALKIS-Integration

---

Auf einen Blick

Rolle	Backend-Entwickler (Umschulung zum Fachinformatiker Anwendungsentwicklung)
Unternehmen	AckerRadar UG (Berliner AgriTech-Startup)
Umfang	IHK-Abschlussprojekt, Teil einer mehrmonatigen Mitarbeit
Stack	Java 17, Spring Boot 3.2, MongoDB (mit Geo-Spatial Indexing), Maven, JWT, OpenAPI 3
Domäne	Präzisionslandwirtschaft, Geo-Daten, Sensor-Integration, Marktdaten-Aggregation
Status	IHK-Abschluss bestanden; Code proprietär (NDA)

Das Problem

Landwirtschaftliche Betriebe in Deutschland arbeiten mit fragmentierten Datenquellen: Wetterdaten kommen aus einem System, Rohstoffpreise aus einem anderen, Bodensensoren aus einem dritten und amtliche Flurstücksdaten (ALKIS) aus einem vierten. Die Konsequenz: Entscheidungen über Düngung, Aussaat und optimale Verkaufszeitpunkte basieren auf manuell zusammengetragenen Daten, oft mit Stunden bis Tagen Verzögerung.

AckerRadar wollte diese Datenquellen in einem System zusammenführen – **mandantenfähig** (mehrere Betriebe parallel auf einer Plattform), **nahezu in Echtzeit** und über **standardisierte REST-APIs** für Frontend-Clients (z.B. QGIS-basierte Karten-Anwendungen) abrufbar.

Meine Aufgabe: die Backend-API entwerfen und implementieren, die diese Aggregation leistet – mit besonderem Fokus auf Plot- (Flurstücks-) Management, Sensor-Daten-Integration und Geo-Spatial Queries.

Anforderungen

Die nicht-funktionalen Anforderungen waren von Beginn an streng:

- **Multi-Tenant-Isolation** – Daten eines Betriebs dürfen niemals für einen anderen sichtbar sein, auch nicht durch fehlerhafte Queries.
- **Geo-Spatial Queries** – „Welche Sensoren liegen innerhalb dieses Flurstücks?“, „Welche Wetterstation liegt meinem Sensor am nächsten?” müssen performant beantwortet werden.
- **Externe API-Integration** – OpenWeatherMap (Wetter), MATIF/Euronext (Getreidepreise), ALKIS (amtliche Flurstücksdaten). Jede Quelle hat eigene Besonderheiten: Rate Limits, Format-Inkonsistenzen, gelegentliche Ausfälle.
- **API-First-Design** – Vollständige OpenAPI-3-Dokumentation, damit Frontend-Teams parallel entwickeln können.
- **JWT-basierte Authentifizierung** mit Tenant-Context-Propagation durch alle Layer.

Architektur

Systemüberblick

Das System folgt einer klassischen Schichtenarchitektur mit klarer Trennung zwischen externen Datenquellen, der Aggregations-API und den konsumierenden Clients.

Externe Datenquellen (OpenWeatherMap, MATIF/Euronext, regionale Getreidebörsen) werden über dedizierte Fetcher- und Scraper-Module angebunden. Die zentrale REST-API stellt die aggregierten Daten authentifizierten Clients zur Verfügung – darunter QGIS-Karten und IoT-Feldsensoren.

Service-Struktur

Innerhalb der API orchestriert ein koordinierender `DataSyncService` die fachlichen Services. Jeder fachliche Service kapselt eine Datendomäne und eine externe Datenquelle:

- `WeatherService` – Anbindung an OpenWeatherMap, Mapping von Sensoren zu nächstgelegenen Wetterstationen
- `MarketDataService` – Aggregation von Börsendaten verschiedener Quellen
- `BlogScraperService` – Web Scraping regionaler Getreidebörsen, deren APIs nicht öffentlich verfügbar sind
- `AuthenticationService` – JWT-Erzeugung und -Validierung

Diese Trennung folgt dem Single-Responsibility-Prinzip und macht jeden Service einzeln testbar und austauschbar.

Datenmodell

Das interessanteste Architektur-Element ist die **atomare Datenstruktur**: Statt Status, Zeitstempel und Position als Felder in jeder Entität zu duplizieren, gibt es drei wiederverwendbare Basis-Komponenten:

- `Distinction` – kapselt Status, Kategorie, Notizen plus Timestamp und optional Location
- `Timestamp` – isolierte Zeitangabe, separat indexierbar
- `Location` – Geokoordinaten plus optionalem GeoJSON-Point

Jede fachliche Entität (Plot, Sensor, SensorData, MarketData, WeatherData usw.) referenziert ein `Distinction`-Objekt, statt eigene Status-Enums und Zeitfelder zu pflegen. Das hat zwei konkrete Vorteile:

1. **Konsistenz** – ein Status-Update folgt überall demselben Pfad. Es gibt keine „ich habe vergessen, `updatedAt` zu aktualisieren”-Bugs mehr.
2. **Audit-Fähigkeit** – jeder Statuswechsel ist automatisch zeitlich nachvollziehbar, ohne separate Audit-Tabelle.

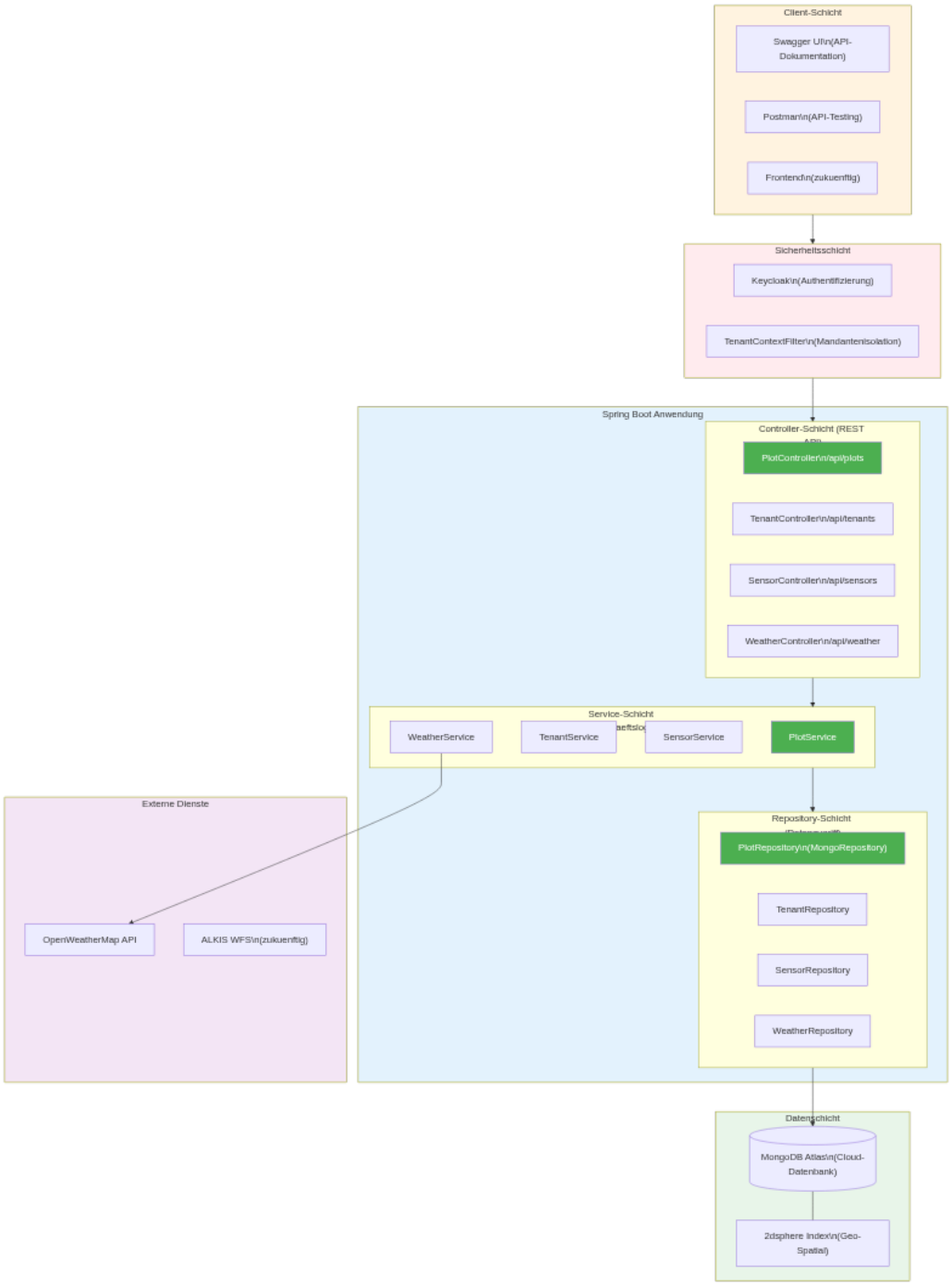

Abbildung 1: *Systemarchitektur AckerRadar*

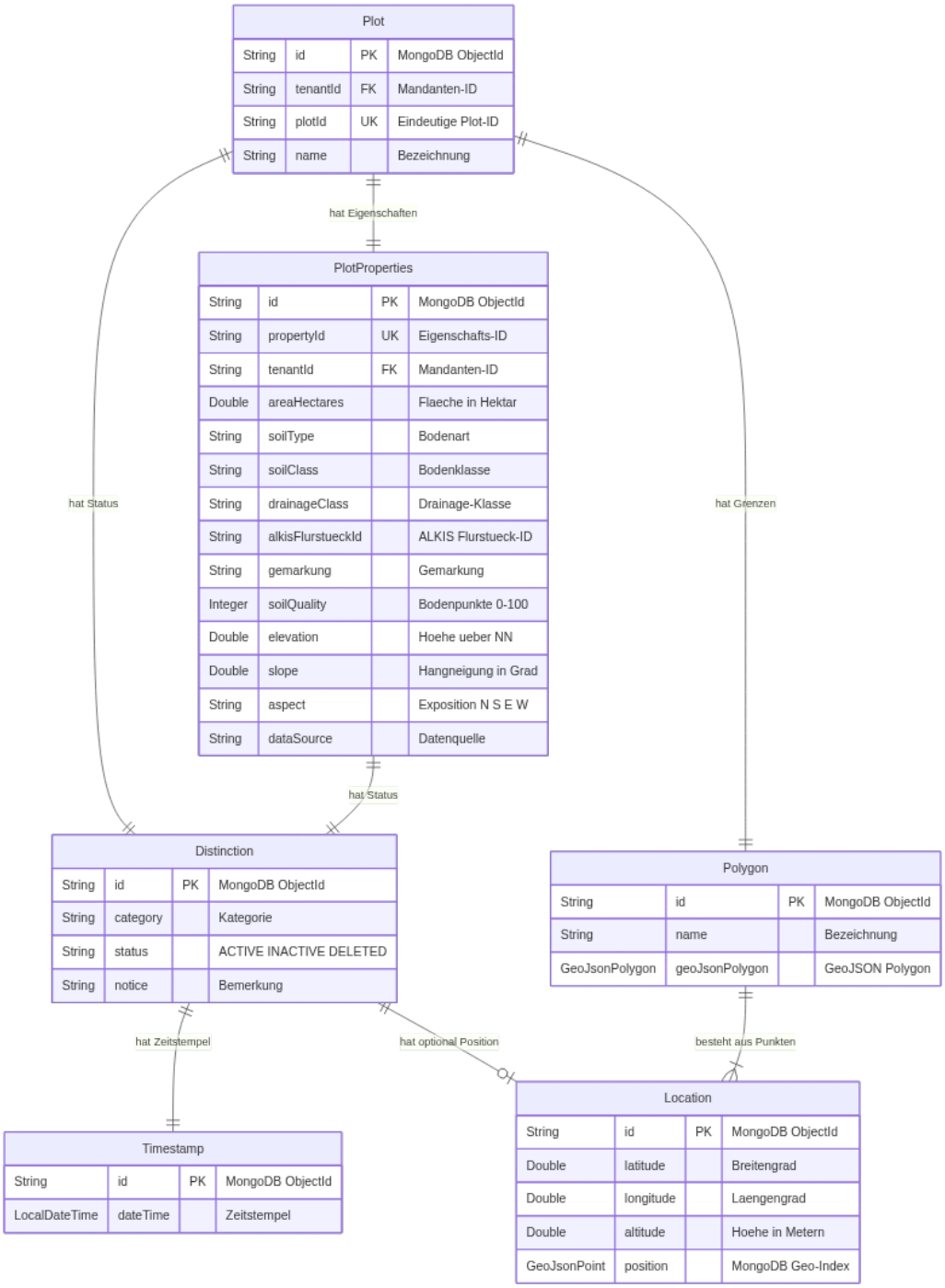

Abbildung 2: *ER-Diagramm – Plot Management System*

Der Trade-off: zusätzliche Indirektion bei Queries und höherer Schreib-Aufwand. In der Praxis hat sich das gelohnt, weil die Audit-Anforderungen im Agrarbereich (Düngeverordnung, Bio-Zertifizierung) nicht verhandelbar sind.

Plot-Management-Klassendiagramm

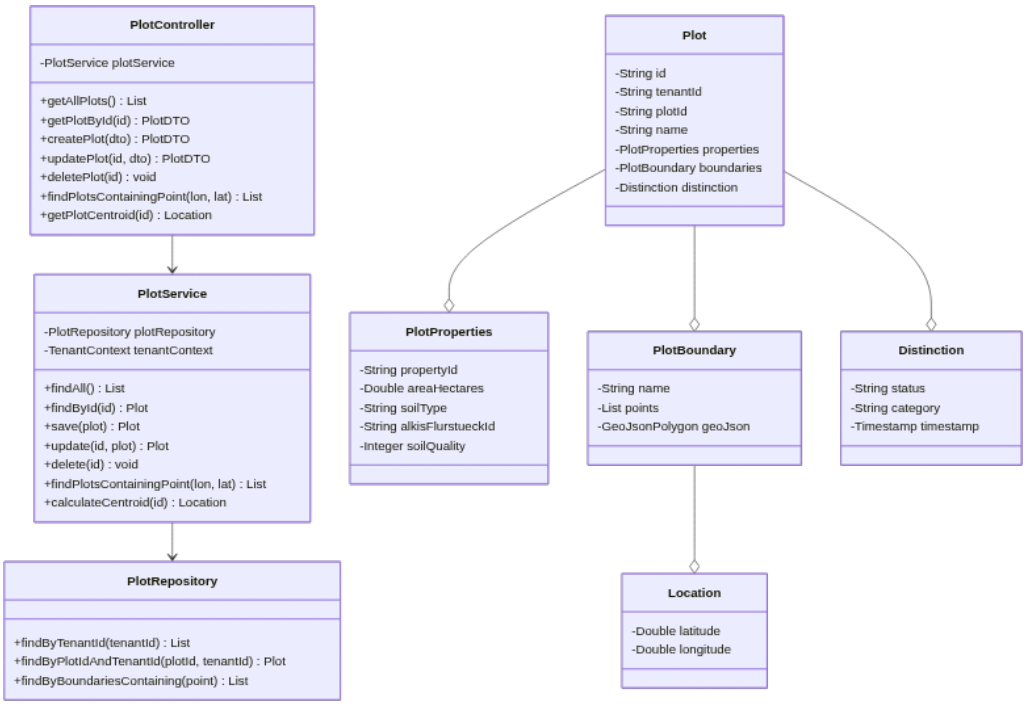

Abbildung 3: *UML-Klassendiagramm – Plot Management*

Ein `Plot` (Flurstück) besteht aus mehreren wiederverwendbaren Komponenten: `PlotProperties` (Bodentyp, Fläche, ALKIS-Referenz), `Polygon` (Geo-Grenzen aus mindestens 3 `Location`-Punkten) und einer `Distinction`. Sensoren werden dem Plot zugeordnet, in dessen Polygon ihre GPS-Koordinaten liegen – nicht durch manuelle Verknüpfung. Das verhindert Inkonsistenzen, wenn Sensoren physisch versetzt werden.

Technische Entscheidungen und Trade-offs

Warum MongoDB statt PostgreSQL/PostGIS

Die ursprüngliche Spezifikation sah MySQL vor. Während des Projekts wechselte das Team auf MongoDB – aus drei Gründen:

1. **Schema-Evolution** – die Sensor-Datenmodelle änderten sich während der Entwicklung mehrfach (von „nur Temperatur und Feuchte" zu „20+ Sensor-Typen inkl. NPK-Werte"). MongoDB erlaubt diese Evolution ohne Migrations-Schmerzen.
2. **Natives Geo-Spatial Indexing** – MongoDB unterstützt 2dsphere-Indizes für GeoJSON nativ, mit Operatoren wie `$geoWithin`, `$nearSphere` und `$geoIntersects`. PostgreSQL hätte PostGIS als Extension erfordert.
3. **Dokumentenstruktur passt zur Domäne** – ein Sensor-Datensatz mit verschachtelten Messwerten, Metadaten und Kalibrierungsinfos ist als Dokument natürlicher als über 5 Tabellen normalisiert.

Der Trade-off: kein echtes ACID über mehrere Collections (vor MongoDB 4.0 gar nicht, seitdem eingeschränkt). Für diese Domäne war das akzeptabel – Sensor-Daten sind append-only, und die wenigen Operationen mit Transaktionsbedarf (z.B. Tenant-Anlage) wurden bewusst sequenziell gestaltet.

Warum Spring Boot statt Javalin

Die Spezifikation sah ursprünglich Javalin vor – ein leichtgewichtiges Web-Framework, das zum anfänglich geplanten Umfang passte. Mit wachsendem Funktionsumfang (Multi-Tenant-Security, JWT, OpenAPI, Repository-Layer, Aggregation Pipelines) wurde Spring Boot zur besseren Wahl: das Ökosystem (Spring Security, Spring Data MongoDB, springdoc-openapi) sparte Wochen an Implementierungs- und Wartungsaufwand. Der Preis: höherer Speicherverbrauch und längere Startzeit – akzeptabel für eine Server-Anwendung, die einmal startet und dann läuft.

DBRef vs. Manual References

In der MongoDB-Welt gibt es zwei Wege, Beziehungen abzubilden: `DBRef` (eingebauter Mechanismus mit automatischer Auflösung) oder manuelle Referenzen (einfache String-IDs). Initial wurden DBRefs verwendet, weil sie „automatischer" wirken. In der Praxis erwiesen sie sich als schwer handhabbar in komplexen Aggregation Pipelines, und Lazy Loading funktionierte nicht zuverlässig mit Tenant-Context-Filter.

Die Lösung: ein hybrider Ansatz. Strukturelle Beziehungen (Plot → PlotProperties, Plot → Polygon) bleiben DBRefs, weil sie meist gemeinsam geladen werden. Tenant-Referenzen (`tenantId`-String in jedem Dokument) sind dagegen denormalisiert – das beschleunigt Tenant-gefilterte Queries deutlich und vereinfacht die Filter-Logik.

Multi-Tenancy

Multi-Tenant-Isolation wurde über drei Mechanismen umgesetzt:

1. **`tenantId` als Pflichtfeld in jedem fachlichen Dokument** – denormalisiert, damit jede Query effizient gefiltert werden kann.
2. **`TenantContext` als ThreadLocal** – pro HTTP-Request wird der Tenant aus dem JWT extrahiert und in einem ThreadLocal abgelegt. Repositories greifen automatisch darauf zu.
3. **Compound-Indizes mit `tenantId` an erster Stelle** – `{tenantId: 1, createdAt: -1}` statt nur `{createdAt: -1}`. Das verhindert, dass MongoDB versehentlich tenant-übergreifend scannt.

Der Trade-off bei ThreadLocals: man muss diszipliniert sein, sie nach jedem Request zu leeren (Memory Leaks im Connection Pool). Das wurde über einen `Filter` in der Spring-Security-Chain gelöst.

Sequenzdiagramm: Plot anlegen

Der Plot-Anlage-Flow zeigt das Zusammenspiel der Layer – inklusive Tenant-Isolation und atomarer Komponenten:

1. Client sendet `POST /api/v1/plots` mit JWT.
2. Spring Security validiert den Token, extrahiert den Tenant und setzt den `TenantContext`.
3. Der Controller delegiert an `PlotService`.
4. `PlotService` erzeugt ein `Distinction`-Objekt mit aktuellem Timestamp.
5. Polygon und PlotProperties werden separat persistiert (eigene Collections, eigene IDs).
6. Der `Plot` selbst referenziert diese per DBRef und wird in der `plots`-Collection gespeichert.
7. Response mit dem vollständig aufgelösten Plot-Objekt geht zurück an den Client.

Herausforderungen und Lösungen

GeoJSON Coordinate Order

GeoJSON spezifiziert `[longitude, latitude]` – die meisten Menschen denken in `[latitude, longitude]`. Spring Data MongoDB folgt der GeoJSON-Spec, OpenWeatherMap mischt beide Konventionen, und ALKIS-Daten kommen in einem weiteren Format. Drei Tage Debugging eines mysteriösen Bugs – Sensoren wurden auf einem Plot in der Nordsee statt in Brandenburg verortet – führten zu einer dedizierten `CoordinateConverter`-Utility mit eindeutigen Methodennamen (`fromLatLon`, `fromGeoJson`).

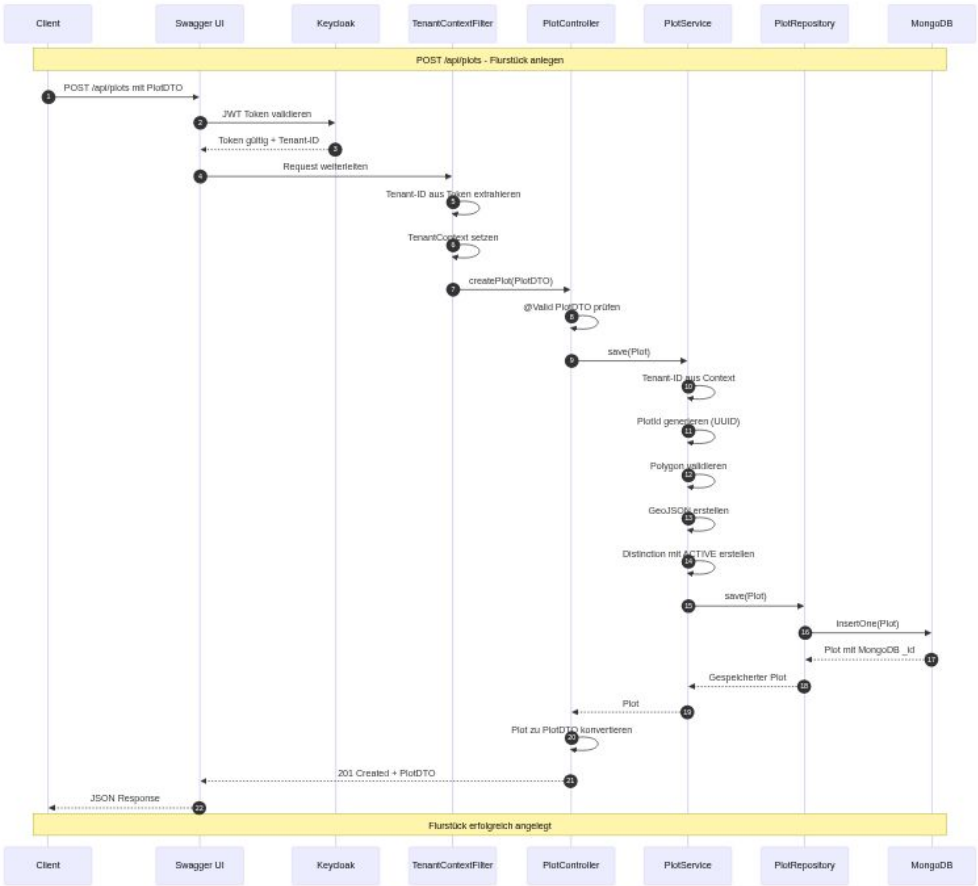

Abbildung 4: *Sequenzdiagramm – Plot erstellen*

Lesson Learned: Wenn ein Format-Konflikt subtil ist, schreibt man den Konverter einmal zentral und macht es danach unmöglich, den Fehler zu wiederholen.

Spring Boot 3 Migration

Während des Projekts erschien Spring Boot 3 mit der Migration von `javax.*` zu `jakarta.*`. Mehrere Dependencies (insbesondere ältere MongoDB-Treiber-Versionen) waren noch nicht kompatibel. Die Lösung: gezieltes Upgrade einzelner Dependencies, kombiniert mit einem schmalen Adapter-Layer für eine kritische Library, die noch keine Jakarta-Version hatte.

MockMvc + Spring Security

Die Testbarkeit der Security-Layer war eine eigene Herausforderung. `@WithMockUser` funktioniert für viele Standardfälle, aber sobald der Tenant-Context aus dem JWT abgeleitet werden muss, braucht es zusätzliche Test-Konfiguration. In der IHK-Dokumentation wurde dieser Punkt bewusst transparent beschrieben – inklusive der Workarounds –, statt eine „alle Tests grün, keine Probleme”-Fassade aufzubauen. Diese Ehrlichkeit wurde im IHK-Fachgespräch positiv aufgenommen.

Ergebnis

- **Voll funktionsfähige REST-API** mit ca. 40 Endpoints, dokumentiert via OpenAPI 3 (Swagger UI).
- **Multi-Tenant-fähig** mit Tenant-Isolation auf Datenbank-, Service- und Controller-Ebene.
- **Geo-Spatial Queries** für Sensor-zu-Plot-Zuordnung, nächstgelegene Wetterstation, Polygon-basierte Plot-Suche.
- **Externe Integrationen** für OpenWeatherMap und ALKIS funktionsfähig; MATIF-Anbindung als Stub vorbereitet.

- **Bestandene IHK-Abschlussprüfung** als Fachinformatiker Anwendungsentwicklung.

Der Code ist proprietär und steht nicht öffentlich zur Verfügung. Diese Case Study dokumentiert Architektur-Entscheidungen und Lessons Learned auf einem Niveau, das ohne Code-Offenlegung auskommt.

Lessons Learned

Atomare Datenstrukturen lohnen sich, wenn Audit-Anforderungen eine Rolle spielen. Die frühe Entscheidung für `Distinction`/`Timestamp`/`Location` als wiederverwendbare Komponenten hat über die gesamte Projektlaufzeit Inkonsistenzen verhindert. In einem Projekt ohne Audit-Anforderungen wäre der Overhead nicht gerechtfertigt – hier war er es.

MongoDB ist eine ausgezeichnete Wahl für Geo-Daten und sich ändernde Schemata, aber kein Drop-in-Replacement für relationale Datenbanken. DBRef-Limitierungen, fehlendes Multi-Document-ACID (vor 4.0) und die Notwendigkeit, Aggregation Pipelines manuell zu optimieren, sind reale Kosten. Die Entscheidung lohnt sich, wenn die Domäne sie rechtfertigt.

Ehrliche Dokumentation schlägt polierte Dokumentation. Technische Probleme (MockMvc-Komplexität, Spring Boot 3 Migration) wurden in der IHK-Dokumentation transparent beschrieben statt kaschiert. Im Fachgespräch funktionierte das besser als jede beschönigte Darstellung.

Multi-Tenancy muss von Tag 1 mitgedacht werden. Tenant-Isolation nachträglich einzuziehen ist deutlich teurer, als von Anfang an `tenantId` in jedem Dokument zu haben und Compound-Indizes entsprechend zu strukturieren.

Über mich

Yves Kühn, Berlin. Quereinsteiger in die Softwareentwicklung über die IHK-Umschulung zum Fachinformatiker Anwendungsentwicklung (bestanden 2026). Schwerpunkte: Java-Backend-Entwicklung, MongoDB, REST-API-Design, zunehmend KI-Integration und AI-Agent-Systeme. Open-Source-Befürworter, Maintainer von **Orbis** (eigene Software, GitHub).

Auf der Suche nach Junior- oder Quereinsteiger-Positionen im Bereich Backend-Entwicklung, Remote oder Berlin.